

Demonstrating the Top-Down Solver in a Box

Marco Cicalè^{1,2}, José F. Morales^{1,2},
Pedro López-García^{1,3}, and Manuel V. Hermenegildo^{1,2}

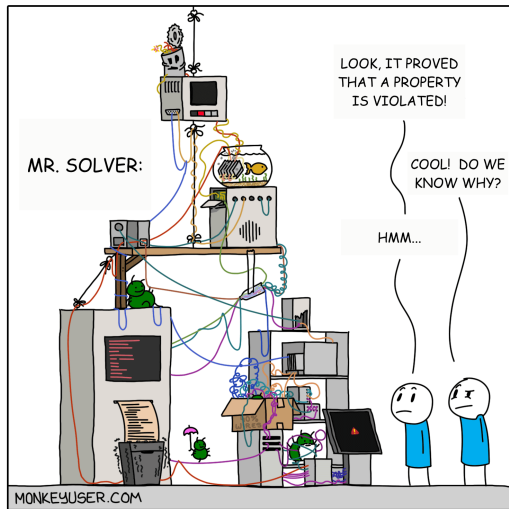
¹ IMDEA Software Institute

² Universidad Politécnica de Madrid (UPM)

³ Spanish Council for Scientific Research (CSIC)

NSAD'26 (SPLASH/ISSTA'26); October 7; Oakland, California, U.S.

Motivation – Do we really need to inspect solvers?



- **Solvers** are complex pieces of software; **abstract interpreters** are no exception.
- **Understanding** them is **crucial**:
 - **actual** program bugs,
 - **imprecise/buggy** domain operations,
 - **many** iterations or huge **abstractions** during the fixpoint computation, etc.
- To do so, we must **inspect** **abstract states** at program points, the **fixpoint computation**, the **domain operations**, etc., including their evolution.

Goal: (interactively) **observe** the solver **while it runs**, at the **granularity** we choose.

(Adapted from: <https://www.monkeyuser.com/2020/next-level/>.)

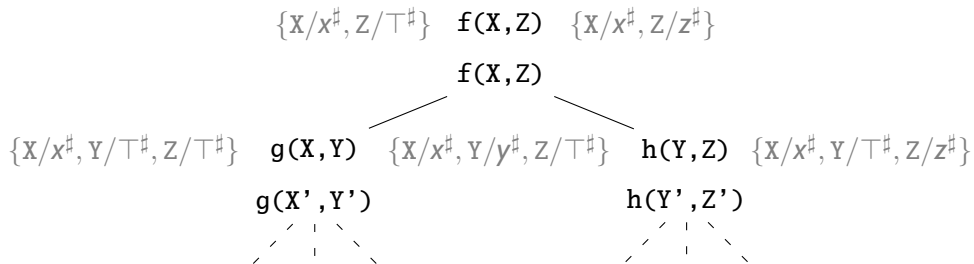
Related work



Tool	What	When
CiaoPP, before [Her+05]	tracing of abstract states and fixpt. ops. & use of traditional debugger sometimes used on the solver code	<i>during analysis</i>
GobPie [Hol+24]	source-level navigation of abstract states	<i>after analysis</i>
Frama-C/EVA [Mas26]	source-level navigation of abstract states	<i>during analysis</i>
Mopsa [MOM24]	gdb-like stepping of analyzer internals (domains/transfer fns.)	<i>during analysis</i>
This work (current CiaoPP)	procedure-box <i>framework</i> : results <i>and</i> internals (states, fixpoint, domain ops.), context-aware, configurable granularity (spy/leash/hide), Emacs, VS Code, Playground	<i>during analysis;</i> <i>user-configurable</i>

Top-down analyzers – PLAI/TD

- **Demand-driven:** from program **entry point(s)**, analyze procedures as they are **encountered**.
- **Context-sensitive:** for each **path to a program point** and each **call abstraction** λ^c at that point, compute a **success abstraction** λ^s ; the result is an **analysis graph**:



- **PLAI/TD:** the *PLAI* (or *top-down solver*) [MH89], at the core of CiaoPP and Goblint.

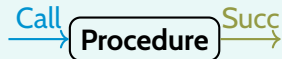
Block-based intermediate representation

Blocks $\Rightarrow$ Rules [MLNH07; Ang+21]		Example	
Block	$\rightarrow$	Rule (Clause)	
Statement	$\rightarrow$	Literal	
Basic command	$\rightarrow$	Built-in call	
Loop	$\rightarrow$	Recursion	
Conditional	$\rightarrow$	Multiple rules	
		<pre>while (i < n) { s += i; i += 1; }</pre>	$\Rightarrow$ <pre>loop(I,N,S0,S) :- I < N, S1 is S0+I, I1 is I+1, loop(I1,N,S1,S). loop(I,N,S,S) :- I >= N.</pre>

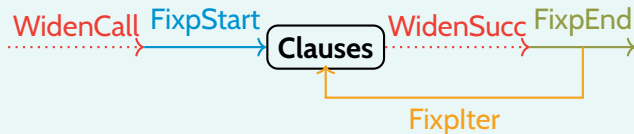
- PLAИ/TD analyzes code in the **block-based** intermediate representation (a.k.a. CHCs).
 - Conditionals, case statements, etc., translated to *alternative, conditional blocks* (clauses).
- This allows PLAИ/TD to analyze many different source languages such as: *logic, functional, object-oriented, smart contracts*, etc. [MH90; MLNH07; PC+20].

Contribution 1: Procedure-box model of PLAI/TD

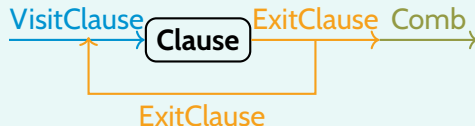
Statement (Literal)



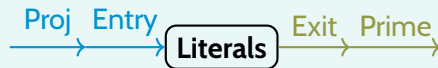
Procedure (Predicate)



Blocks (Clauses)

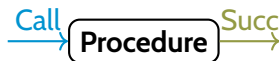


Block (Clause)



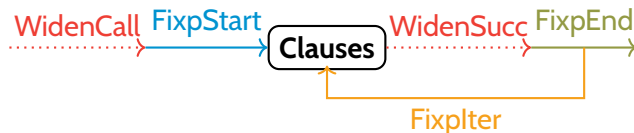
- One **box** with multiple **ports** per layer of the algorithm.
- Each port represents an **observable event**.

Procedure-box model of PLAI/TD – The Statement (Literal) box



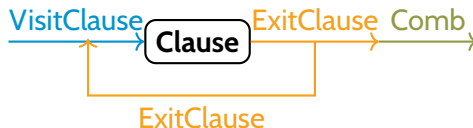
- **Call**: Fires when the analysis of a statement (literal) with call abstraction λ^c is about to start; control passes to the **Procedure (Predicate)** box.
 - (Built-ins/language constructs treated similarly to procedure calls.)
- **Succ**: Fires once the *fixpoint* is *stable*, carrying the success abstraction λ^s .

Procedure-box model of PLAI/TD – The Procedure (Predicate) box



- **FixpStart**, **Fixplter**, **FixpEnd**:
Fire at the *beginning*, *each iteration*, and the *end* of the *fixpoint* computation.
→ Carry the **iteration count**.
- **WidenCall**, **WidenSucc**: May fire to *widen* the call and success abstractions.
→ Carry the **widening operands**.

Procedure-box model of PLAI/TD – The Blocks (Clauses) box



- All the *alternative blocks* (clauses) of a procedure are analyzed **unconditionally**.
- **VisitClause**, **ExitClause**: Fire when **starting** and **finishing** the analysis of a block (clause).
→ Carry the **block (clause) identifier**.
- **Comb**: Fires when the analysis results of **all** alternative blocks (clauses) of a procedure are **combined**.

Procedure-box model of PLAI/TD – The Block (Clause) box



- **Proj, Entry**: Fire when *projecting* λ^c onto the procedure call's (literal's) variables, then onto the clause's head variables (block entry point).
→ Carry the **variable bindings** between both contexts.
- Block body **statements** (clause **literals**), *left to right*: built-ins via the abstract transfer function; procedure calls re-enter the **Statement (Literal)** box.
- **Exit, Prime**: Fire upon termination of the analysis of the body statements (literals), projected back into the calling context.
→ Carry the **variable bindings** between both contexts.

Contribution 2: Debugger for PLAI/TD – Observables as messages

Every *port event* emits one message:

[+] <inv-id> <node-id> <port> : <lit> : <abs> <info> [?]

- <inv-id>: unique invocation ID.
- <node-id>: analysis-graph node.
- <port>: port name.
- <lit>: the literal under analysis.
- <abs>: variables abstract state.
- [+]: break-point marker.
- [?]: interactive prompt.

Ports	<info>
Proj, Entry, Exit, Prime	variables abstract state
FixpStart, FixpIter, FixpEnd	iteration count
WidenCall, WidenSucc	widening operands
VisitClause, ExitClause	block/clause identifier

Example:

+ 7 2 Call: h(Y,Z): [X/x[#], Y/y[#], Z/⊥[#]] ?

Debugger for PLAI/TD – Three levels of granularity control

Event level

what happens next

- **Break-points.**
- **Creep:** next event.
- **Skip:** step over a call.
- **Leap:** next break-point.

Trace level

what is shown

- **Hide** ports.
- Filter events of interest: domain operations, fixpoint computation, ...

Interaction level

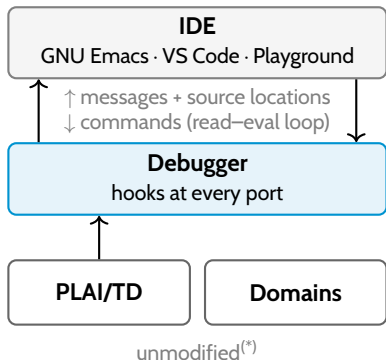
where analysis stops

- **Unleash** ports.
- Events are printed, not stopped at.
- Debugger + tracer.



Other tools (e.g., LLMs): consume a filtered, non-interactive trace in a uniform format, or directly drive the debugger.

Debugger for PLAI/TD – Architecture and integration



- **Thin instrumentation layer** over PLAI/TD in Ciao: intercepts events via hooks at each port.
- New events: **extend** this layer and add the corresponding **hooks** in the solver.^(*)
- Source locations $\Rightarrow$ **highlighting** in the host IDE.
- Configured **on the fly** through a read-eval loop.

Use case 1 – Non-interactive monitoring

Consider:

- **qplan**: a query planner program. 330 LoC, 37 procedures; rules with many variables and complex sharing patterns.
- **Share+Cliques** set-sharing domain [JL89; MH89; NBH06]; relational abstraction of **sets of variables** that may **share** a common *substructure* (e.g., aliasing). Worst case: **exponential**.
- Analysis time: **7.6s**.

Hypothesis: The solver performs too many fixpoint iterations.

Setup:

- *Non-interactive* mode: all ports *unleashed* (i.e., the debugger acts as a tracer).
- Only show fixpoint ports: **FixpStart**, **FixpIter**, **FixpEnd**.

Use case 1 – Non-interactive monitoring (cont'd)

Only fixpoint ports:

```
1466 21 FixpEnd: mark(P,L,Q0,Q1): ⟨abs⟩ (iter 3)
3427 46 FixpEnd: plan(Q2,V,C,G,Q3): ⟨abs⟩ (iter 2)
3875 98 FixpEnd: variablise(T,V0,V1): ⟨abs⟩ (iter 4)
```

⇒ At most **4 iterations** per procedure: hypothesis **rejected**.

Also clause-level ports:

```
3421 80 Succ: strip_keys(L0,L1):
    [clique([[L0,L1,L2,L3,L4,L5,L6,L7,L8]]),
    sharing([[P,L],[P,L,Q0],[P,L,Q0,Q1],[P,L,Q0,Q1,Q2],...]])]
```

⇒ **1024 sharing sets**, each with up to 12 variables: the **abstractions** are the bottleneck.

Fix: abstract environment trimming [Jur+24], which shrinks abstractions as the analysis progresses:

```
3421 80 Succ: strip_keys(L0,L1): [sharing([[L0],[L1],[L0,L1],[L0,L1,P],...]])]
```

⇒ From 1024 to **64** sharing sets, and from 7.6s to **0.8s** of analysis time.

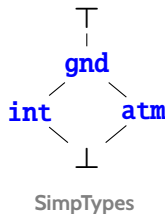
Use case 2 – Faulty abstract domain implementation

Consider implementing a simple domain **SimpTypes**, and verifying:

```
:- pred lookup_name/2 : atm * term => atm * gnd.
```

```
lookup_name(Name, Id) :- db(Id, Name, ...).
```

```
lookup_name(Name, no).
```



Expected: $\text{Id} \mapsto \text{gnd} \Rightarrow$ either an integer (**int**) or an atom (**atm**).

Got: $\text{Id} \mapsto \top$ (**term**) $\Rightarrow$ **assertion not verified!**

Suspect: How the clause results are *combined* (**Comb** event).

Setup:

- Break-point on lookup_name/2.
- Only show **Call**, **Prime**, **Comb**.
- Only lease **Comb**.

DEMO!

Use case 3 – Understanding proofs

Consider verifying a naive implementation of the list-reverse procedure:

```
:- pred nrev/2 : list(num) * var => list(num) * list(num) + det.
```

```
nrev(    [], []).
```

```
nrev([H|L0], L2) :- nrev(L0, L1), conc(L1, [H], L2).
```

Property	Domain	
var	Share+Cliques : alias pointer	[NBH06]
list (num)	ETerms : parametric types	[VB02]
det	NfDet : non-failure + determinism (hyperproperties)	[DLGH97; LGBH05]

The assertion is **verified**.

Question: How does each domain contribute to the proof?

DEMO!

Summary

- **Unified model** (procedure-box ports on PLAI/TD) covers all three uses:
 - abstract debugging,
 - fixpoint monitoring, and
 - domain debugging.
- Observables at **several layers**: source, abstract states, fixpoint and widening events, domain operations (e.g., Prime, Comb).
- **Configurable** granularity, with Byrd-style controls: spy-points and breakpoints, creep/leap, port hiding, leashing and unleashing.
- **Context-aware**: shows how procedures are analyzed separately for each calling context.
- Front ends: Emacs, VS Code and the web Playground.
- Use cases illustrated through examples.

References I

- [Ang+21] E.D. Angelis et al. “Analysis and Transformation of Constrained Horn Clauses for Program Verification”. In: **Theory and Practice of Logic Programming** 22.6 (2021), pp. 1–69. ISSN: 1471-0684. DOI: 10.1017/S1471068421000211. URL: <https://arxiv.org/abs/2108.00739>.
- [DLGH97] S.K. Debray, P. Lopez-Garcia, and M.V. Hermenegildo. “Non-Failure Analysis for Logic Programs”. In: **1997 International Conference on Logic Programming**. Cambridge, MA: MIT Press, Cambridge, MA, 1997, pp. 48–62.
- [Her+05] M.V. Hermenegildo et al. “Integrated Program Debugging, Verification, and Optimization Using Abstract Interpretation (and The Ciao System Preprocessor)”. In: **Science of Computer Programming** 58.1–2 (2005), pp. 115–140. ISSN: ISSN 0167-6423. DOI: 10.1016/j.scico.2005.02.006.
- [Hol+24] K. Holter et al. “Abstract Debuggers: Exploring Program Behaviors using Static Analysis Results”. In: **Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software**. Onward! '24. Pasadena, CA, USA: Association for Computing Machinery, 2024, 130–146. ISBN: 9798400712159. DOI: 10.1145/3689492.3690053. URL: <https://doi.org/10.1145/3689492.3690053>.
- [JL89] D. Jacobs and A. Langen. “Accurate and Efficient Approximation of Variable Aliasing in Logic Programs”. In: **1989 North American Conference on Logic Programming**. MIT Press, 1989.
- [Jur+24] Daniel Jurjo et al. “Abstract Environment Trimming”. In: **Theory and Practice of Logic Programming** 24.4 (2024), pp. 863–884. ISSN: 1471-0684. DOI: 10.1017/S1471068424000358. URL: <https://doi.org/10.1017/S1471068424000358>.

References II

- [LGBH05] P. Lopez-Garcia, F. Bueno, and M.V. Hermenegildo. “Determinacy Analysis for Logic Programs Using Mode and Type Information”. In: **Proceedings of the 14th International Symposium on Logic-based Program Synthesis and Transformation (LOPSTR’04)**. LNCS 3573. Springer-Verlag, 2005, pp. 19–35.
- [Mas26] J. Massart. **The Eva Abstract Debugger: A new way to explore and interact with the Frama-C/Eva static analyzer**. <https://www.frama-c.com/2026/02/02/Eva-abstract-debugger.html>. (Visited on July 2026). 2026.
- [MH89] K. Muthukumar and M.V. Hermenegildo. “Determination of Variable Dependence Information at Compile-Time Through Abstract Interpretation”. In: **1989 North American Conference on Logic Programming**. MIT Press, 1989, pp. 166–189.
- [MH90] K. Muthukumar and M.V. Hermenegildo. **Deriving A Fixpoint Computation Algorithm for Top-down Abstract Interpretation of Logic Programs**. Technical Report ACT-DC-153-90. Microelectronics and Computer Technology Corporation (MCC), Austin, TX 78759, 1990. URL: <https://cliplab.org/papers/mcctr-fixpt.pdf>.
- [MLNH07] M. Méndez-Lojo, J. Navas, and M.V. Hermenegildo. “A Flexible (C)LP-Based Approach to the Analysis of Object-Oriented Programs”. In: **17th International Symposium on Logic-based Program Synthesis and Transformation (LOPSTR 2007)**. Lecture Notes in Computer Science 4915. Springer-Verlag, 2007, pp. 154–168. DOI: 10.1007/978-3-540-78769-3_11.

References III

- [MOM24] R. Monat, A. Ouadjaout, and A. Miné. “Easing maintenance of academic static analyzers”. In: **International Journal on Software Tools for Technology Transfer** 26.6 (2024), pp. 673–686. DOI: 10.1007/s10009-024-00770-1. URL: <https://doi.org/10.1007/s10009-024-00770-1>.
- [NBHO6] J. Navas, F. Bueno, and M.V. Hermenegildo. “Efficient Top-Down Set-Sharing Analysis Using Cliques”. In: **8th International Symposium on Practical Aspects of Declarative Languages (PADL’06)**. LNCS 2819. Springer-Verlag, 2006, pp. 183–198.
- [PC+20] V. Perez-Carrasco et al. “Cost Analysis of Smart Contracts via Parametric Resource Analysis”. In: **Proceedings of the 27th Static Analysis Symposium (SAS 2020)**. Ed. by David Pichardie and Mihaela Sighireanu. Vol. 12389. LNCS. Keynote at SAS’2020. Video available at SPLASH site. Springer, 2020, pp. 7–31. DOI: 10.1007/978-3-030-65474-0_2. URL: <https://cliplab.org/papers/resources-blockchain-sas20.pdf>.
- [VBO2] C. Vaucheret and F. Bueno. “More Precise yet Efficient Type Inference for Logic Programs”. In: **9th International Static Analysis Symposium (SAS’02)**. Vol. 2477. Lecture Notes in Computer Science. Springer-Verlag, 2002, pp. 102–116.

Demonstrating the Top-Down Solver in a Box

Marco Cicalè^{1,2}, José F. Morales^{1,2},
Pedro López-García^{1,3}, and Manuel V. Hermenegildo^{1,2}

¹ IMDEA Software Institute

² Universidad Politécnica de Madrid (UPM)

³ Spanish Council for Scientific Research (CSIC)

NSAD'26 (SPLASH/ISSTA'26); October 7; Oakland, California, U.S.

Use case 2 – The trace (backup)

Break-point on lookup_name/2; only **Call**, **Prime** and **Comb** shown; only **Comb** leased:

```
+ 1 1 Call: lookup_name(Name0,Id0): [atm(Name0),top(Id0)] ?  
...  
+ 25 2 Prime: lookup_name(Name1,Id1): [atm(Name1),int(Id1)] with [Name1=Name0,Id1=Id0] ?  
...  
+ 33 2 Prime: lookup_name(Name1,Id1): [atm(Name1),atm(Id1)] with [Name1=Name0,Id1=Id0] ?  
+ 34 2 Comb: lookup_name(Name1,Id1): [atm(Name1),top(Id1)] with [Name1=Name0,Id1=Id0] ?
```

- Clause 1: $\text{Id1} \mapsto \text{int}$ Clause 2: $\text{Id1} \mapsto \text{atm}$.
- Combined: $\text{int} \sqcup \text{atm} = \text{T}$, but it should be **gnd**.

⇒ The program is fine: the **lub of the domain** is wrong.

◀ Back to the talk

Use case 2 – The fix (backup)

The culprit is `lub_el/3: int` and `atm` are incomparable, so they fall through to the last clause.

```
lub_el(int, atm, gnd) :- !.  
lub_el(atm, int, gnd) :- !.  
lub_el(X, Y, Y) :- less_or_equal_el(X, Y), !.  
lub_el(X, Y, X) :- less_or_equal_el(Y, X), !.  
lub_el(_, _, top).
```

The fix: two clauses for the missing case.

Re-analyzing:

```
+ 34  2 Comb: lookup_name(Name1,Id1): [atm(Name1),gnd(Id1)] with [Name1=Name0,Id1=Id0] ?
```

⇒ The assertion is **verified**.

[◀ Back to the talk](#)

Use case 2 – In GNU Emacs, after the fix (backup)

```
5 :- module(lookup, [lookup_name/2], [assertions]).
4
3 :- pred lookup_name/2 : atm * term => atm * gnd.
2
1 lookup_name(Name, Id) :- db(Id, Name, ...).
6 lookup_name(Name, no).
```

```
{Analyzing (simp_types): [.../absdbg/examples/lookup.pl]
{init for the plai fixpoint in 0.011 msec.}
{preprocessed for the plai fixpoint in 0.129 msec.}
+ 1 1 Call: lookup_name(Name0,Id0): [atm(Name0),top(Id0)] ?
...
+ 25 2 Prime: lookup_name(Name1,Id1): [atm(Name1),int(Id1)]
      with [Name1=Name0,Id1=Id0] ?
...
+ 33 2 Prime: lookup_name(Name1,Id1): [atm(Name1),atm(Id1)]
      with [Name1=Name0,Id1=Id0] ?
+ 34 2 Comb: lookup_name(Name1,Id1): [atm(Name1),gnd(Id1)]
      with [Name1=Name0,Id1=Id0] ?
```

lookup.pl 6:21 Top

[Compiling] LF UTF-8 Ciao (+3)

Ciao 14886:104 Bot

[Compiling] LF UTF-8 Ciao Listener (+3) : run

[← Back to the talk](#)

Use case 3 – ETerms (backup)

Interactive mode; only **Call**, **Succ** and fixpoint ports shown:

```
...  
+ 28 1 FixpIter: nrev(L,L2): [rt7(L),rt6(L2)] (iter 2)  
      where {  
          rt7([]). rt7([A|B]) :- num(A), rt8(B).  
          rt8([]). rt8([A|B]) :- num(A), rt8(B).  
          rt6([]). rt6([A]) :- num(A).  
      } ? 1  
+ 32 1 Call: nrev(L,L2): [list(num,L),term(L2),num(H),term(L1)] ? 1  
+ 33 1 Succ: nrev(L,L2): [rt7(L),rt6(L2),term(L1),num(H)] ?
```

⇒ L and L2 are described by **regular types**: at iteration 2, rt6 only covers lists of **at most one** number; it grows until the fixpoint.

[◀ Back to the talk](#)

Use case 3 – NfDet (backup)

Leap to NfDet; only **VisitClause**, **Prime** and **Comb** shown:

```
+ 15 13 VisitClause: nrev(L,L2): [...] (clause nrev:nrev/2/1) ? 1
+ 21 13 Prime: nrev(L,L2): [ground([L,L2]),rt82(L),rt83(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    with [L=[],L2=[]] where { ... } ? 1
...
+ 140 13 VisitClause: nrev(L,L2): [...] (clause nrev:nrev/2/2) ? 1
+ 199 13 Prime: nrev(L,L2): [ground([L,L2]),rt163(L),rt157(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    with [L=[H|L0],L2=L3] where { ... } ? 1
+ 201 13 Comb: nrev(L,L2): [ground([L,L2]),rt89(L),rt132(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])] ?
```

⇒ covered: some clause always applies (no failure); mut_exclusive: at most one applies (**det**). The assertion is **verified**.

[← Back to the talk](#)

Use case 3 – In GNU Emacs (backup)

```
7 :- module(nrev, [nrev/2], [assertions, nativeprops]).
6
5 :- pred nrev/2 : list(num) * var
4         => list(num) * list(num) + det.
3
2 nrev( [], []).
1 nrev([H|L0], L2) :- nrev(L0,L1), conc(L1,[H],L2).
8 []
1 conc( [], L, L).
2 conc([H|L0], L1, [H|L2]) :- conc(L0,L1,L2).

...
+ 15 13 VisitClause: nrev(L,L2): [sharing([[L2]]),free([L2]),
    ground([L]),list(num,L),term(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    (clause nrev:nrev/2/1) ? 1
+ 21 13 Prime: nrev(L,L2): [ground([L,L2]),rt82(L),rt83(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    with [L=[],L2=[]] where { rt82([]). rt83([]). } ? 1
...
+ 140 13 VisitClause: nrev(L,L2): [sharing([[L2]]),free([L2]),
    ground([L]),list(num,L),term(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    (clause nrev:nrev/2/2) ? 1
+ 199 13 Prime: nrev(L,L2): [ground([L,L2]),rt163(L),rt157(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])]
    with [L=[H|L0],L2=L3] where {
    rt163([A|B]) :- num(A), rt89(B).
    rt157([A|B]) :- num(A), rt158(B).
    rt158([]). rt158([A|B]) :- num(A), rt159(B).
    rt159([]). rt159([A|B]) :- num(A), rt159(B). } ? 1
+ 201 13 Comb: nrev(L,L2): [ground([L,L2]),rt89(L),rt132(L2),
    det([L,L2]),covered([L,L2]),mut_exclusive([L,L2])] ?
```

① nrev.pl 8:0 All

[Compiling] LF UTF-8 Ciao (+3)

① *Ciao* 15851:134 Bot

[Compiling] LF UTF-8 Ciao Listener (+3) : ru

◀ Back to the talk